

# Project: Network Packet Analysis with Wireshark

**Objective:** To gain practical experience in capturing, filtering, and analyzing network traffic using Wireshark, and to understand the fundamental concepts of various network protocols.

## Part 1: Analyzing HTTP Traffic (3.0 pt.)

### Instructions:

1. **Open Your Capture File:** start a new capture and repeat the steps to visit <http://example.com>.
2. **Apply a Display Filter:** In the filter bar at the top of Wireshark, type `http` and press Enter. This will filter the displayed packets to show only HTTP traffic.
3. **Identify HTTP Requests and Responses:** Look for `GET` requests (your browser asking for a page) and `200 OK` responses (the server sending the page).
4. **Examine Packet Details:** Select an HTTP `GET` request packet and expand the "Hypertext Transfer Protocol" layer in the Packet Details pane. Do the same for an HTTP `200 OK` response packet.
5. **Observe TCP Connections:** Now, try filtering for the TCP stream associated with your HTTP conversation.
  - Select an HTTP packet.
  - Right-click on it and choose `Follow > TCP Stream`. This will open a new window showing the raw data of the TCP stream and automatically apply a filter like `tcp.stream eq X` (where X is the stream index). Close this new window after reviewing.
  - Observe the flow of TCP packets (SYN, ACK, PSH, FIN) around your HTTP requests and responses.

### Questions for Analysis (Part 1):

1. Locate an HTTP GET request. What information is contained in the "Host:" header?
2. Locate an HTTP 200 OK response. What is the "Content-Type:" header, and what does it indicate?
3. Based on the HTTP traffic, can you tell if the website is using a secure connection (HTTPS)? Why or why not?
4. Persistent vs. Non-Persistent Connections:
  - a. After observing the TCP streams associated with your HTTP traffic to `example.com`, do you believe your browser used a **persistent** or **non-persistent** HTTP connection? Provide evidence from the Wireshark capture to support your answer. Specifically, look for the header in HTTP responses, or analyze the TCP connection setup and teardown for multiple HTTP requests.
  - b. Briefly explain the advantage of using persistent HTTP connections over non-persistent ones for web Browse.
5. Transfer-Encoding: Chunked:
  - a. In an HTTP `200 OK` response, look for the `Transfer-Encoding` header. Is it present, and if so, what is its value?

b. If **Transfer-Encoding: chunked** is present, briefly explain why a web server might use chunked encoding to send a response. What is its primary benefit, especially when the total size of the response body is not known in advance?

---

## Part 2: Investigating DNS Traffic (2.0 pt.)

### Instructions:

1. **Start a New Capture:** Start a new live capture in Wireshark.
2. **Clear DNS Cache:** Before Browse, it's good practice to clear your system's DNS cache to ensure fresh DNS queries are made.
  - **Windows:** Open Command Prompt as administrator and type `ipconfig /flushdns`.
  - **macOS/Linux:** Open Terminal and type `sudo dscacheutil -flushcache` (macOS) or `sudo systemctl restart systemd-resolved` (Linux, depending on your distro).
3. **Browse to a New Website:** In your web browser, navigate to a website you haven't visited recently (e.g., <https://www.google.com> or <https://www.nasa.gov>).
4. **Stop Capture:** Stop the Wireshark capture after the page has fully loaded.
5. **Apply a Display Filter:** In the filter bar, type `dns` and press Enter.
6. **Examine DNS Queries and Responses:** Look for DNS query packets (Type A for IPv4 addresses) and their corresponding DNS response packets.

### Questions for Analysis (Part 2):

1. Find a DNS Query packet. What is the "Query type" and "Query name" in the packet details?
  2. Find the corresponding DNS Response packet. What is the "Answer" section displaying?
  3. What transport layer protocol (TCP or UDP) is primarily used by DNS, and why?
  4. Did you see any DNS queries for the website you visited in Part 1 ([example.com](https://www.example.com))? Explain why or why not.
- 

## Part 3: Exploring TCP and UDP Protocols (3.0 pt.)

### Instructions:

1. **Start a New Capture:** Begin a new live capture in Wireshark.
2. **Generate TCP Traffic:**
  - Open your web browser and navigate to a complex website (e.g., an online news site with many images and scripts). Let it load fully.
  - Download a small file (e.g., a PDF document).
3. **Generate UDP Traffic (Optional, if possible):**
  - If you have a Voice over IP (VoIP) application (like Skype or Zoom), make a short call.
  - *Alternatively*, if your network allows, try playing an online game or streaming a short video. These often use UDP for real-time data.
4. **Stop Capture:** Stop the Wireshark capture.

### 5. Apply Filters:

- Filter for TCP traffic: `tcp`
- Filter for UDP traffic: `udp`

### Questions for Analysis (Part 3):

1. What is the key difference between TCP and UDP?
2. **For TCP traffic:**
  - Find a TCP handshake (SYN, SYN-ACK, ACK). Describe the purpose of each of these flags.
  - Identify the source and destination port numbers for a typical web Browse TCP connection. What are these port numbers commonly associated with?
  - Look for TCP segments with the `FIN` or `RST` flag set. What do these flags indicate?
  - In the "Flags" section of a TCP packet, what do the "Window size value" and "Calculated window size" fields represent?
3. **For UDP traffic (if captured):**
  - Find a UDP packet. Are there any handshake mechanisms similar to TCP?
  - What information is available in a UDP header compared to a TCP header?
  - Based on your capture, for what types of applications or services do you observe UDP traffic?

---

## Part 4: Troubleshooting Scenario (2.0 pt.)

**Scenario:** A user reports that they are unable to access a specific website (e.g., [www.badwebsite.com](http://www.badwebsite.com) - *replace with a real, safe, blocked website if you have one available, or simulate a blocked site by modifying your hosts file to point it to a non-existent IP*). You suspect a network connectivity issue or a firewall blocking.

### Instructions:

1. **Start a New Capture:** Begin a fresh Wireshark capture.
2. **Attempt to Access the "Bad" Website:** In your web browser, try to navigate to [www.badwebsite.com](http://www.badwebsite.com).
3. **Stop Capture:** Stop the Wireshark capture shortly after the attempt.
4. **Analyze the Capture:** Use various filters and analysis techniques to diagnose the problem. Think about what kind of traffic you *expect* to see and what you *actually* see.

### Questions for Analysis (Part 4):

1. What steps did you take to attempt to diagnose the issue using Wireshark?
2. What specific protocols or packets did you look for? (e.g., DNS, TCP, ICMP, HTTP)
3. What was the most significant packet or series of packets that indicated the problem? (e.g., DNS lookup failed, TCP connection reset, no HTTP response)
4. Based on your analysis, what do you believe is the root cause of the user's inability to access [www.badwebsite.com](http://www.badwebsite.com)? Provide specific evidence from your packet capture.

5. Suggest a possible solution or next troubleshooting step based on your findings.

---

## Submission Requirements:

For each part of the project, students should submit:

- **Screenshots:** Relevant screenshots of your Wireshark interface showing captured packets, applied filters, and expanded packet details as requested by the questions.
- **Answers to Questions:** Clear and concise answers to all analysis questions for each part.
- **Saved Capture Files:** submit the saved .pcapng files.
- Example: Submit a <StudentID1-StudentID2-StudentID3>.zip file, which is created by zipping a folder named <StudentID1-StudentID2-StudentID3>, structured as follows:

<StudentID1-StudentID2-StudentID3>/      <-- main folder (named with student IDs)

|

| — report.pdf      <-- lab report

| — packets/      <-- packets folder

| | — part1.pcapng

| | — part2.pcapng

| | — part3.pcapng

| | — part4.pcapng

---

## Evaluation Criteria:

- **Accuracy:** Correct identification and interpretation of packet data.
- **Completeness:** All questions answered thoroughly.
- **Clarity:** Answers are well-explained and easy to understand.
- **Technical Understanding:** Demonstration of a solid grasp of network protocols and Wireshark functionality.
- **Troubleshooting Process (Part 4):** Logical and systematic approach to problem-solving.